Algorytmy blokowe - laboratorium

Implementacja algorytmu Blowfish

Agata Wójtowicz

January 27, 2026

Abstract

Przedstawiono dwie implementacje algorytmu Blowfish zaprojektowanego przez Bruce'a Schneiera. Implementacje wykonano na podstawie opisu wykonanego w ramach Internet Engineering Task Force (IETF) przez autorów: Bruce'a Schneier'a oraz Peter'a Gutmann'a [1]. Do testów wykorzystano wektory testowe zamieszczone na stronie autora algorytmu [2]. Aby przyspieszyć implementację algorytmu, wykorzystano techniki optymalizacji przy stałej budowie samego algorytmu, ponieważ budowa Blowfish nie pozwala na wiele modyfikacji.

1 Wpowadzenie

Parametry algorytmu:

Rozmiar bloku: 64 bity

Liczba rund: 16

Rozmiar klucza: 0d 32 do 488 bitów

Implementacja algorytmu składa się z dwóch głównych części:

I: Inicjalizacja

II: Szyfrowanie, Deszyfrowanie

Algorytm wyróżnia się wykorzystaniem S-boxów zależnych od klucza użytkownika. Do wygenerowania kluczy wyorzystywane są zarówno tablice S-box, tablice P-array oraz funkcja szyfrująca. To sprawia, że generowanie kluczy jest bardzo kosztowną czasowo częścią algorytmu, ale zwiększa jego bezpieczeństwo.
Funkcja f oraz funcja rundy zapewniają: konfuzję poprzez podstawienie za pomocą tablicy S-boxów oraz dyfuzję poprzez wykorzystanie 4 różnych Sboxów w funkcji f oraz operację zamiany dwóch słów 32-bitowych w każdej rundzie i wielokrotnego przejścia bloku przez funkcję f (zgodnie ze strukturą sieci Feistela).

2 Implementacja referencyjna

2.1 Inicjalizacja tablic

Do inicjalizacji tablic został wykorzystany plik nagłówkowy (Załącznik B - ab_lab.h). Blowfish wykorzystuje tablice S-box[4][256] (w progamie S_INIT) czterech słów 256 bitowych oraz P-array[18] (w proramie P_INIT) złożoną z 18 słów 32 bitowych z których następnie generowane jest 18 kluczy rundowych. Wartości w tablicach umieszczonych w pliku nagłówkowym są stałe i pochodzą z rozwinięcia liczby pi. Pierwszym krokiem algorytmu jest wygenerowanie struktury S-boxów oraz P-array zależnych od klucza użytkownika. Na tablicy P-array jest wykonywana operacja xor z kluczem użytkownika. Następnie na obie tablice są szyfrowane z wykorzystaniem wartości wczytanych z tablic. Krokiem początkowym jest szyfrowanie bloku 64-bitowego złożonego z samych zer, w kolejnych krokach szyfrowane są wartości z tablic w blokach 64-bitowych. W ten sposób uzyskuje się tablice zależne od klucza użytkownika. Blowfish jest algorytmem projektowanym z myślą o szyfrowaniu danych z wykorzystaniem stałych kluczy, ponieważ to proces generowania kluczy rundowych oraz wartości początkowych algorytmu (tablic S[4][256] oraz P[18]) jest najbardziej

kosztowny czasowo. Stąd, w referenyjnej impmentacji (wolniejszej) założono, że inicjalizacja tablic danymi z pliku będzie odbywać się bit po bicie. W celu przetestowania wydajności, wymuszono za pomocą zmiennej typu volatile, aby optymalizacja przez wczytywanie całych wartości 32-bitowych nie była wykonywana.

2.1.1 Pseudokod

Algorithm 1 Wczytanie wartości z tablic - implementacja referencyjna

```
P-array
for i = 0 to (N + 1) do
    target ← 0
    source ← P_INIT [i]
    for bit = 0 to bit = 32 do
        mask ← (1 ≪ bit)
        if (source & mask) is true then
            target ← target ∨ mask
        end if
    end for
    P[i] ← target
end for

S-box
for i = 0 to i = 3 do
    for j = 0 to j = 256 do
        target ← 0
        source ← S_INIT[i][j]
        for bit = 0 to bit = 31 do
            mask ← (1 ≪ bit)
            if (source & mask) is true then
                target ← target ∨ mask
            end if
        end for
        S[i][j] ← target
    end for
end for
```

Algorithm 2 Algorytm inicjalizacji tablic P[18] oraz S[4][256] zależnych od klucza

```
P-array
keyIndex = 0
for i = 0 to 17 do
    value ← 0
    for j = 0 to 3 do
        value ← (value ≪ 8)|Key [keyIndex++]
        keyIndex %= len( Key )
    end for
    P[i] ^= data
end for

dataL ← 0x00000000
dataR ← 0x00000000
for i to (N + 1) do
    for i = 0 to (N + 1) do
        encrypt(dataL, dataR)
        P[i] ← dataL
        P[i + 1] ← dataR
    end for
end for

S-box
dataL ← 0x00000000
dataR ← 0x00000000
for i = 0 to 3 do
    for i = 0 to i = 3 do
        encryptBlock(dataL, dataR)
        S[i][j] ← dataL
        S[i][j + 1] ← dataR
    end for
end for
```

2.1.2 Kod źródłowy

```c
void blowfishInitialize(uint8_t *key, int keyLen) {
    int i, j, k;
    uint32_t data;
    uint32_t datal = 0x00000000;
    uint32_t datar = 0x00000000;

    // 1. Inicjalizacja P-array (kopiowanie bit po bicie)
    for (i = 0; i < N + 2; ++i) {
        uint32_t target = 0;
        uint32_t source = P_INIT[i];
        for (int bit = 0; bit < 32; ++bit) {
            // volatile wymusza brak optymalizacji
            volatile uint32_t mask = (1 << bit);
            if (source & mask) {
                target |= mask;
            }
        }
        P[i] = target;
    }

    // 2. Inicjalizacja S-box w (kopiowanie bit po bicie)
    for (i = 0; i < 4; ++i) {
        for (j = 0; j < 256; ++j) {
            uint32_t target = 0;
            uint32_t source = S_INIT[i][j];
            for (int bit = 0; bit < 32; ++bit) {
                volatile uint32_t mask = (1 << bit);
                if (source & mask) {
                    target |= mask;
                }
            }
```

```
            S[i][j] = target;
        }
    }

    // 3. XORowanie klucza z P-array
    j = 0;
    for (i = 0; i < N + 2; ++i) {
        data = 0x00000000;
        for (k = 0; k < 4; ++k) {
            data = (data << 8) | key[j];
            j++;
            if (j >= keyLen) j = 0;
        }
        P[i] = P[i] ^ data;
    }

    // 4. Szyfrowanie zer i nadpisywanie P oraz S
    datal = 0x00000000;
    datar = 0x00000000;

    for (i = 0; i < N + 2; i += 2) {
        encryptBlock(&datal, &datar);
        P[i] = datal;
        P[i + 1] = datar;
    }

    for (i = 0; i < 4; ++i) {
        for (j = 0; j < 256; j += 2) {
            encryptBlock(&datal, &datar);
            S[i][j] = datal;
            S[i][j + 1] = datar;
        }
    }
}
```

Listing 1: Inicalizacja Blowfish z wymuszonym wczytywaniem po jednym bicie

2.2 Klucze rundowe

Klucze rundowe znajdują się w tablicy P_[18] opisanej podczas inicjalizacji parametrów algorytmu. Na początku klucz użytkownika musi zostać dopełniony zerami, tak aby był długości conajmniej 32 bitów. Wartości początkowe z tablicy P_INIT są xorowna z kluczem użytkownika. Jeżeli wykorzystywany klucz jest dłuższy niż 32 bity, najpierw xorowane jest pierwsze 32 bitów klucza, a potem cyklicznie kolejne 32-bitowe bloki klucza. Klucze rundowe wykorzystywane w funkcji rundy zapisywane są do tablicy P[18] zainicjalizowanej na początku programu.

2.3 Funcja F

Funkcja F wykonywana jest na słowie 32-bitowym (x). Słowo dzielone jest na cztery 8-bitowe fragmenty: (a, b, c, d). Uzyskane wartości to indeksy S-boxów, których wartości są ze sobą mieszane za pomocą operacji dodawania oraz operacji xor.
Funkcja F zwraca wartość uint32_t. Mimo, że szyfrowane są bloki 64-bitowe, to zgodnie z opisaną w następnej części funkcji rundy, funkcja F jest wykonywana na słowie 32-bitowym. Dodatkowo w implementacji wykorzystywane są przesunięcia bitowe, dlatego wszystkie operacje wykonywane są na liczbach bez znaku.

2.3.1 Pseudokod

Algorithm 3 Funkcja F - Blowfish

Require: x - argument 32-bitowy
Ensure: y - "zaszyfrowany" argument 32-bitowy
Podział wejścia
d ← bity od 0 do 7
c ← bity od 8 do 15
b ← bity od 16 do 23
a ← bity od 24 do 31
Pobranie wartości z tablicy S[4][256]
a ← indeks z tablicy S[0]
b ← indeks z tablicy S[1]
c ← indeks z tablicy S[2]
d ← indeks z tablicy S[3]
Mieszanie pobranych wartości
y ← S[0][a] + S[1][b]
y ← y ^= S[2][c]
y ← y + S[3][d]

2.3.2 Kod źródłowy

```
uint32_t F(uint32_t x) {
    uint8_t a, b, c, d;
    uint32_t y;

    // Podzia  32-bitowego s owa na cztery 8-bitowe cz   ci
    d = x & 0xFF;
    x >>= 8;
    c = x & 0xFF;
    x >>= 8;
    b = x & 0xFF;
    x >>= 8;
    a = x & 0xFF;

    // Operacje na S-boxach: ((S[0][a] + S[1][b]) ^ S[2][c]) + S[3][d]
    y = S[0][a] + S[1][b];
    y = y ^ S[2][c];
    y = y + S[3][d];

    return y;
}
```

Listing 2: Funkcja F algorytmu Blowfish

2.4 Funkcja rundy

Pojedyncza runda składa się z trzech operacji. Blok 64-bitowy jest dzielony na dwie części (oznaczane jako L i R), na połowie bloku wykonywana jest funkcja F(x) a na połowie operacja xor z odpowiednim kluczem rundowym. Ostatnią operacją jest zamiana bloków miejscami. W procesie szyfrowania naprzemiennie wykonywana jest funkcja rundy nieparzysta i parzysta:

oddRoundEncrypt:

L ^= P [numerRundy - 1]

R ^= F(L)

evenRoundEncrypt:

R ^= P [numerRundy - 1]

L ^= F(R)

Algorytm składa się z 16 takich rund, do których wykorzystywane jest 16 kluczy rudnowych. Na końcu części bloku L i R są zamieniane oraz dodawane są dwa ostanie klucze rundowe. W

implementacji, 16 rund jest wykonywanych w pętli, w której przy każdej iteracji nasteuje zamiana stron. Ostatnia zamiana wykonywana jest poza pętlą:

Algorithm 4 Szyfrowanie - Blowfish

```
L ← dataL
R ← dataR
for i = 0 to (N-1) do
    L ← L ^ P[i]
    R ← F(L) ^ R
    swap(L, R)
end for
swap(L, R)
R ← R ^ P[16]
L ← L ^ P[17]                                   ▷ =0
```

Funkcja deszyfrująca działa analogicznie do funkcji szyfrującej, jedyna różncia polega xorowaniu kluczy rundowych o indeksach w odwrotnej kolejności.

```c
void encryptBlock(uint32_t *xl, uint32_t *xr) {
    uint32_t L = *xl;
    uint32_t R = *xr;
    uint32_t temp;
    int i;

    // G   wna p tla szyfruj ca (16 rund)
    for (i = 0; i < N; i++) {
        L = L ^ P[i];
        R = F(L) ^ R;

        // Zamiana stron (Swap)
        temp = L;
        L = R;
        R = temp;
    }

    // Cofni cie ostatniej zamiany (Undo swap)
    temp = L;
    L = R;
    R = temp;

    // Ko cowe wybielanie (Whitening)
    R = R ^ P[N];
    L = L ^ P[N + 1];

    *xl = L;
    *xr = R;
}

void decryptBlock(uint32_t *xl, uint32_t *xr) {
    uint32_t L = *xl;
    uint32_t R = *xr;
    uint32_t temp;
    int i;

    // Deszyfrowanie u ywa kluczy w odwrotnej kolejno ci
    for (i = N + 1; i > 1; i--) {
        L = L ^ P[i];
        R = F(L) ^ R;

        // Zamiana stron
        temp = L;
        L = R;
        R = temp;
    }

    // Cofni cie ostatniej zamiany
    temp = L;
    L = R;
    R = temp;

    // Ko cowe wybielanie odwrotnymi kluczami
    R = R ^ P[1];
```

```
    L = L ^ P[0];

    *xl = L;
    *xr = R;
}
```

Listing 3: Implementacja szyfrowania i deszyfrowania bloku (Blowfish)

3 Optymalizacja

3.1 Wykonane zmiany

3.1.1 Wczytywanie tablic

W implementacji referencyjnej wykorzystana została bardzo niefektywna czasowo metoda wczytywania tablic z pliku nagłówkowego - początkowych wartości w tablicach SINIT oraz P_INIT. Pomimo, że wartości początkowe są wczytywane tylko raz na początku, następnie przekształcane, a później wykorzystywane do szyfrowania bloków przez cały czas, to zmiana wykonana w tej części programu przyniosła największ różnice w uzyskanym czasie działania.
Tablica P_INIT to 576 bitów, a S_INIT to 32 768 bitów, co daje 33 344 odwołań do pamięci w przypadku kopiowaia danych bit po bicie. Algorytm Blowfish był projektowany z myślą o procesorach 32-bitowych, dlatego licząc słowa 32 bitowe w obu tablicach otrzymujemy 1042 odwołania. W funkcji inicjalizującej zmieniono jedynie sposób kopiowania początkowych danych z tablic, pozostała logika pozostaje bez zmian. Tablice są szyfrowane tak samo jak w implementacji referencyjnej.

```
void blowfishInitialize(uint8_t *key, int keyLen) {
    int i, j, k;
    uint32_t data;
    uint32_t datal = 0x00000000;
    uint32_t datar = 0x00000000;

    // 1. Kopiowanie sta ych inicjalizacyjnych
    memcpy(P, P_INIT, sizeof(P));
    memcpy(S, S_INIT, sizeof(S));

    // 2. XORowanie P-array z kluczem (cyklicznie)
    j = 0;
    for (i = 0; i < N + 2; ++i) {
        data = 0x00000000;
        for (k = 0; k < 4; ++k) {
            data = (data << 8) | key[j];
            j++;
            if (j >= keyLen) j = 0;
        }
        P[i] = P[i] ^ data;
    }

    // 3. Generowanie kluczy dynamicznych (szyfrowanie zer)
    datal = 0x00000000;
    datar = 0x00000000;

    // Nadpisywanie P-array
    for (i = 0; i < N + 2; i += 2) {
        encryptBlock(&datal, &datar);
        P[i] = datal;
        P[i + 1] = datar;
    }

    // Nadpisywanie S-box w
    for (i = 0; i < 4; ++i) {
        for (j = 0; j < 256; j += 2) {
            encryptBlock(&datal, &datar);
            S[i][j] = datal;
            S[i][j + 1] = datar;
        }
    }
}
```

Listing 4: Inicjalizacja Blowfish - memcpy

3.1.2 Funkcja F jako makro

Funkcja F jest wywoływana:

16 razy podczas szyfrowania jednego bloku

$9 \cdot 16 = 144$ razy podczas inicjizalizacji tablicy P[18] (9 wywołań i 16 rund szyfrowania)

$4 \cdot 128 \cdot 16 = 8192$ razy podczas inicjalizowania tablicy S[4][256] (4 tablice po 128 wywołań i 16 rund szyfrowania)

Zamiana funkcji typu uint32_t na makro pozwala na optymalizację czasu każdorazowego wywoływania funkcji poprzez stos. Zapisanie funkcji w taki sposób jest możliwe, dzięki temu, że jest to operacja arytmetyczna, która przyjmuje jeden argument na wejściu i zwraca jeden argument na wyjściu, zmienne pomocnicze wykorzystywane w funkcji z implementacji refernecyjnej (a, b, c, d) mogą zostać zastąpione przesunięciami bitowymi.

```
#define F(x) ( \
    ((S[0][((x) >> 24) & 0xFF] + S[1][((x) >> 16) & 0xFF]) ^ \
      S[2][((x) >> 8)  & 0xFF]) + S[3][(x) & 0xFF]          \
)
```

Listing 5: Funkcja F jako makro)

3.1.3 Rozwinięcie pętli szyfrującej do postaci poszczególnych rund

Nie zmieniając struktury wykonywania 16 rund szyfrowania, rozpisano jawnie każdą z iteracji w kodzie zamiast wykorzystaia pętli. Ta modyfikacja wydłuża kod wynikowy i zajmuje więcej miejsca w pamięci programu. Celem tego rozwiązania jest:

- Skrócenie czasu o operacje związane z wykonywaniem pętli: inkrementacje, porównanie warunku, skok warunkowy
- Przyspieszenie adresowanie zainicjalizowanych tablic (zamiast obliczania adresu, od razu uzyskanie dostępu do konkretnej wartości)

```
void encryptBlock(uint32_t *xl, uint32_t *xr) {
    uint32_t L = *xl;
    uint32_t R = *xr;
    uint32_t temp;

    // Runda 0
    L ^= P[0];
    R ^= F(L);
    temp = L; L = R; R = temp;

    // Runda 1
    L ^= P[1];
    R ^= F(L);
    temp = L; L = R; R = temp;

    // Runda 2
    L ^= P[2];
    R ^= F(L);
    temp = L; L = R; R = temp;

    // Runda 3
    L ^= P[3];
    R ^= F(L);
    temp = L; L = R; R = temp;

    // Runda 4
    L ^= P[4];
    R ^= F(L);
    temp = L; L = R; R = temp;

    // Runda 5
    L ^= P[5];
    R ^= F(L);
    temp = L; L = R; R = temp;

    // Runda 6
    L ^= P[6];
```

```
    R ^= F(L);
    temp = L; L = R; R = temp;

    // Runda 7
    L ^= P[7];
    R ^= F(L);
    temp = L; L = R; R = temp;

    // Runda 8
    L ^= P[8];
    R ^= F(L);
    temp = L; L = R; R = temp;

    // Runda 9
    L ^= P[9];
    R ^= F(L);
    temp = L; L = R; R = temp;

    // Runda 10
    L ^= P[10];
    R ^= F(L);
    temp = L; L = R; R = temp;

    // Runda 11
    L ^= P[11];
    R ^= F(L);
    temp = L; L = R; R = temp;

    // Runda 12
    L ^= P[12];
    R ^= F(L);
    temp = L; L = R; R = temp;

    // Runda 13
    L ^= P[13];
    R ^= F(L);
    temp = L; L = R; R = temp;

    // Runda 14
    L ^= P[14];
    R ^= F(L);
    temp = L; L = R; R = temp;

    // Runda 15
    L ^= P[15];
    R ^= F(L);
    temp = L; L = R; R = temp;

    temp = L; L = R; R = temp;

    R ^= P[16];
    L ^= P[17];

    *xl = L;
    *xr = R;
}
```

Listing 6: Rozwinięcie pętli)

Uwagi do metody Pomimo teoretycznych podstaw do wykorzystania tej metody, w praktyce nie wpłynęła ona na efetkywność programu, a jedynie zwiększyła wykorzystanie pamięci. Ponieważ pętla wykonuje się 16 razy, koszt wykonania pętli nie jest duży, ponadto wszystkie wymienione "operacje pętli" mogą zostać wykonane równolegle z obliczeniami funkcji rundy. Ta zmiana wykonana została jedynie w celu przetestowania, najlepszych możliwych wyników dla wykonanej implementacji [3].

3.2 Analiza budowy algorytmu Blowfish

Wprowadzono zmiany w celu optymalizacji implementacyjnej programu. W celu zmniejszenia złożoności obliczeniowej i uzyskania zauważalnie różnych rezultatów w efektywności, należało wykonać optymalizację na poziomie samego algorytmu. Jednak budowa algorytmu Blowfish wykluczyła rozważane metody.

1. Zrównoleglenie rund podczas szyfrowania
   W algorytmie Blowfish wyjście z jednej rundy jest wejściem do kolejnej rundy. Z uwagi na

modyfikowanie naprzemiennie dwóch części bloku funkcją F połączenie dwóch rund nie jest niemożliwe.

2. Wykorzystanie odczytu z tablic
   W algorytmie Blowfish ten sposób optymalizacji został częściowo wprowadzony. W funkcji F, wejście 32-bitowe jest dzielone na mniejsze bloki, a następnie odpowiednia wartość jest odczytywana z tablicy S[4][256]. Wykonywanie jednej dużej tablicy dla całego wejścia 32-bitowego, wymagałoby tablicy 2^{32} elementowej, co nie zoptymalizowałoby implementacji.

3. Generowanie kluczy rundowych
   Wysoka złożoność i długa inicjalizacja są założeniami algorytmu [2]. Wykonanie inicjalizacji tablic (czyli najdłuższej części algorytmu) jest wymagane do wykonania pierwszej rundy szyfrowania. Geneorwanie kluczy w trakcie szyfrowania dla kolejnych rund nie jest możliwe.

4 Testy poprawności i szybkości

Wykonano testy sprawdzające poprawność oraz czas szyfrowaniu. W tym celu wykorzystano 34 wektory testowe zamieszczone w dokumentacji algorytmu [2]. Pomiary czasu powtórzono pięciokrotnie na tym samym zestawie wektorów.

Table 1: Wektory testowe

Key	**Plaintext**	**Ciphertext**
0000000000000000	0000000000000000	4EF997456198DD78
FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF	51866FD5B85ECB8A
3000000000000000	1000000000000001	7D856F9A613063F2
1111111111111111	1111111111111111	2466DD878B963C9D
0123456789ABCDEF	1111111111111111	61F9C3802281B096
1111111111111111	0123456789ABCDEF	7D0CC630AFDA1EC7
0000000000000000	0000000000000000	4EF997456198DD78
FEDCBA9876543210	0123456789ABCDEF	0ACEAB0FC6A0A28D
7CA110454A1A6E57	01A1D6D039776742	59C68245EB05282B
0131D9619DC1376E	5CD54CA83DEF57DA	B1B8CC0B250F09A0
07A1133E4A0B2686	0248D43806F67172	1730E5778BEA1DA4
3849674C2602319E	51454B582DDF440A	A25E7856CF2651EB
04B915BA43FEB5B6	42FD443059577FA2	353882B109CE8F1A
0113B970FD34F2CE	059B5E0851CF143A	48F4D0884C379918
0170F175468FB5E6	0756D8E0774761D2	432193B78951FC98
43297FAD38E373FE	762514B829BF486A	13F04154D69D1AE5
07A7137045DA2A16	3BDD119049372802	2EEDDA93FFD39C79
04689104C2FD3B2F	26955F6835AF609A	D887E0393C2DA6E3
37D06BB516CB7546	164D5E404F275232	5F99D04F5B163969
1F08260D1AC2465E	6B056E18759F5CCA	4A057A3B24D3977B
584023641ABA6176	004BD6EF09176062	452031C1E4FADA8E
025816164629B007	480D39006EE762F2	7555AE39F59B87BD
49793EBC79B3258F	437540C8698F3CFA	53C55F9CB49FC019
4FB05E1515AB73A7	072D43A077075292	7A8E7BFA937E89A3
49E95D6D4CA229BF	02FE55778117F12A	CF9C5D7A4986ADB5
018310DC409B26D6	1D9D5C5018F728C2	D1ABB290658BC778
1C587F1C13924FEF	305532286D6F295A	55CB3774D13EF201
0101010101010101	0123456789ABCDEF	FA34EC4847B268B2
1F1F1F1F0E0E0E0E	0123456789ABCDEF	A790795108EA3CAE
E0FEE0FEF1FEF1FE	0123456789ABCDEF	C39E072D9FAC631D
0000000000000000	FFFFFFFFFFFFFFFF	014933E0CDAFF6E4
FFFFFFFFFFFFFFFF	0000000000000000	F21E9A77B71C49BC
0123456789ABCDEF	0000000000000000	245946885754369A
FEDCBA9876543210	FFFFFFFFFFFFFFFF	6B5C5A9C5D9E0A5A

4.1 Wyniki - implementacja referencyjna

Table 2: Wyniki implementacji referencyjnej

Iteracja testów	Liczba poprawnych testów	Czas iteracji [ms]	Czas 1 testu [ms]
1	34	18	0.530
2	34	17	0.500
3	34	20	0.588
4	34	21	0.618
5	34	16	0.471
Średni czas iteracji [ms]			18.4
Średni czas jednego testu [ms]			0.541
Oszacowana liczba bloków na sekundę			1847.83

4.2 Wyniki - implementacja zoptymalizowna

Table 3: Wyniki implementacji zoptymalizowanej

Iteracja testów	Liczba poprawnych testów	Czas iteracji [ms]	Czas 1 testu [ms]
1	34	2	0.059
2	34	2	0.059
3	34	3	0.088
4	34	2	0.059
5	34	2	0.059
Średni czas iteracji [ms]			2.2
Średni czas jednego testu [ms]			0.065
Oszacowana liczba bloków na sekundę			15454.55

5 Wnioski - porównanie

1. Średni czas szyfrowania bloku został skrócony około ośmiokrotnie. Dla każdej z implementacji wykonano szyfrowanie bloku 64-bitowego 170 razy. Na podstawie uzyskanych wyników obliczono, że korzystając ze zoptymalizowanej implementacji w sekundę można zaszyfrować o 13 607 bloków tekstu więcej.

2. Największy wpływ na czas wykonywania programu miał sposób kopiowania danych z tablic zainicjalizowanych do tablic, na których wykonywane jest szyfrowanie wartości przy pomocy klucza.

5.1 Porównanie wykorzystania pamięci

Pamięć dla zmiennych globalnych - niezależenie od implementacji stałe tablice P_INIT oraz S_INIT zajmują 4168 bajtów

Funkcje szyfrujące i deszyfrujące (encryptBlock, decryptBlock) - w zależności od implementacji znacznie różnią się rozmiarem w pamięci. Dla kodu referencyjnego około 200 bajtów, dla kodu przyspieszonego około 16 razy więcej do 2.5 KB

Funkcja inicjalizująca - implementacja referencyjna (około 200 bajtów) wymaga około 20 razy więcej pamięci niż przyspieszona z wykorzystaniem mempcy (jedynie wywołanie funkcji - 10 bajtów). Niezależnie od implementacji, tablice S-box i P-array zainicjalizownane z kluczem użytkownika zajmują kolejne 4168 bajtów.

Podsumowując, implementacja referencyjna wymaga około 8KB pamięci nieulotnej, a przyspieszona 11 KB. Pamięć RAM nie zmienia się dla obu implementacji i wynosi 4.2 KB, głównie przez duże tablice S-boxów.

References

[1] Peter Gutmann and Bruce Schneier. *Description of the Blowfish Cipher*. IETF Datatracker, 28 Aug. 1999. Dostępny: https://datatracker.ietf.org/doc/html/draft-schneier-blowfish-00.

[2] Bruce Schneier. *The Blowfish Encryption Algorithm*. Schneier on Security, 24 Aug. 2020. Dostępny: https://www.schneier.com/academic/blowfish/

[3] A. Khalid, M. Hassan, G. Paul, and A. Chattopadhyay. RunFein: a rapid prototyping framework for Feistel and SPN-based block ciphers. *Journal of Cryptographic Engineering*, 6(4):299–323, 2016. DOI: https://doi.org/10.1007/s13389-016-0116-7.